

▼ PASO 1 — Limpieza y organización

```
from google.colab import files
import pandas as pd

uploaded = files.upload()
filename = next(iter(uploaded))
df = pd.read_excel(filename)
df.head()
```

Choose files PM TOTALES CU.xlsx

PM TOTALES CU.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 55046 bytes, last modified: 20/08/2025 - 100% done

Saving PM TOTALES CU.xlsx to PM TOTALES CU (1).xlsx

|   | MOD   | CARRERA                                                       | 2025A<br>AS | 2025A<br>AD | 2025A<br>PM | 2024B<br>AS | 2024B<br>AD | 2024B<br>PM | 2024A<br>AS | 2024A<br>AD | ... | 2011B<br>PM | 2011A<br>AS | 2011A<br>AD | 2011A<br>PM |
|---|-------|---------------------------------------------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-----|-------------|-------------|-------------|-------------|
| 0 | CUAAD | LICENCIATURA<br>EN<br>ARQUITECTURA                            | 760         | 220         | 156.35      | 1061        | 220         | 156.3500    | 671         | 220         | ... | 165.1111    | 713.0       | 135.0       | 162.4189    |
| 1 | CUAAD | LICENCIATURA<br>EN DISEÑO,<br>ARTE Y<br>TECNOLOGIAS<br>INT... | 268         | 40          | 176.3367    | 368         | 40          | 176.3367    | 300         | 40          | ... | NaN         | NaN         | NaN         | NaN         |
| 2 | CUAAD | LICENCIATURA<br>EN DISEÑO DE<br>INTERIORES Y<br>AMBIENT...    | 207         | 60          | 148.13      | 268         | 60          | 148.1300    | 217         | 60          | ... | 158.6622    | 322.0       | 60.0        | 156.5556    |
| 3 | CUAAD | LICENCIATURA<br>EN DISEÑO DE<br>MODAS                         | 62          | 40          | 135.92      | 42          | 40          | 135.9200    | 49          | 40          | ... | 154.1100    | 66.0        | 40.0        | 144.6667    |
| 4 | CUAAD | LICENCIATURA<br>EN DISEÑO<br>INDUSTRIAL                       | 330         | 80          | 154.75      | 454         | 80          | 154.7500    | 277         | 80          | ... | 159.4356    | 302.0       | 75.0        | 157.5156    |

5 rows × 95 columns

```
# Renombrar columnas principales
df.columns = [col.strip() for col in df.columns]
df.columns.values[0:2] = ['CU', 'CARRERA']

# Eliminar filas completamente vacías
df.dropna(how='all', inplace=True)

# Derretir columnas por periodo
value_vars = df.columns[2:]
df_melted = df.melt(id_vars=["CU", "CARRERA"], value_vars=value_vars,
                    var_name="PERIODO_TIPO", value_name="VALOR")

# Separar PERIODO y TIPO (AS, AD, PM)
df_melted[['PERIODO', 'TIPO']] = df_melted['PERIODO_TIPO'].str.extract(r'(\d{4}[AB])\s+(\w+)', expand=True)

# Reorganizar a columnas normales
df_limpio = df_melted.pivot_table(
    index=["CU", "CARRERA", "PERIODO"], columns="TIPO", values="VALOR", aggfunc='first'
).reset_index()
```

```
# Renombrar columnas
df_limpio = df_limpio.rename(columns={"AS": "ASPIRANTES", "AD": "ADMITIDOS", "PM": "PUNTAJE_MINIMO"})

# Extraer año y ciclo
df_limpio["AÑO"] = df_limpio["PERIODO"].str.extract(r'(\d{4})').astype(int)
df_limpio["CICLO"] = df_limpio["PERIODO"].str.extract(r'([AB])')

# Convertir a valores numéricos
for col in ["ASPIRANTES", "ADMITIDOS", "PUNTAJE_MINIMO"]:
    df_limpio[col] = pd.to_numeric(df_limpio[col], errors="coerce")

# Limpiar texto
```

```
df_limpio["CU"] = df_limpio["CU"].astype(str).str.strip().str.upper()
df_limpio["CARRERA"] = df_limpio["CARRERA"].astype(str).str.strip().str.upper()

# Copia principal de trabajo
df_carreras = df_limpio.copy()
```

```
# Tasa de Demanda (TD) y Tasa de Admisión (TA)
df_carreras["TASA_DEMANDA"] = df_carreras["ASPIRANTES"] / df_carreras["ADMITIDOS"]
df_carreras["TASA_ADMISION"] = df_carreras["ADMITIDOS"] / df_carreras["ASPIRANTES"]

# Ordenar para cálculo de variaciones
df_carreras.sort_values(["CARRERA", "PERIODO"], inplace=True)

# Cambio porcentual de aspirantes por carrera
df_carreras["CAMBIO_ASPIRANTES"] = df_carreras.groupby("CARRERA")["ASPIRANTES"].pct_change()
```

```
df_ciclo_A = df_carreras[df_carreras["CICLO"] == "A"].copy()
df_ciclo_B = df_carreras[df_carreras["CICLO"] == "B"].copy()
```

```
# Preparar datos con pesos (admitidos)
df_peso = df_carreras[["CU", "PERIODO", "PUNTAJE_MINIMO", "ADMITIDOS"]].dropna()
df_peso["PESO"] = df_peso["ADMITIDOS"]

# Función de ponderación
def calcular_ponderado(grupo):
    puntaje_pond = (grupo["PUNTAJE_MINIMO"] * grupo["PESO"]).sum() / grupo["PESO"].sum()
    return pd.Series({"PUNTAJE_MINIMO": puntaje_pond})

# DataFrame de resumen por CU y periodo
df_resumen_cu = (
    df_peso.groupby(["CU", "PERIODO"], group_keys=False)[["PUNTAJE_MINIMO", "PESO"]]
    .apply(calcular_ponderado)
    .reset_index()
)
```

## ✓ PASO 2 — Exploración y visualizaciones avanzadas (EDA)

Descripción....

### > SECCIÓN 1. Comparativas temporales y globales

↳ 8 cells hidden

### > SECCIÓN 2. Comparativa entre ciclos A y B

↳ 7 cells hidden

### ✓ SECCIÓN 3. Análisis por CU (Centro Universitario)

#### 3.1 Heatmap de aspirantes por CU y año

 **Objetivo:** Visualizar la intensidad de demanda (número de aspirantes) a lo largo del tiempo por cada centro universitario.

 **Tipo de gráfico:** Heatmap (mapa de calor)

 **Variables clave:**

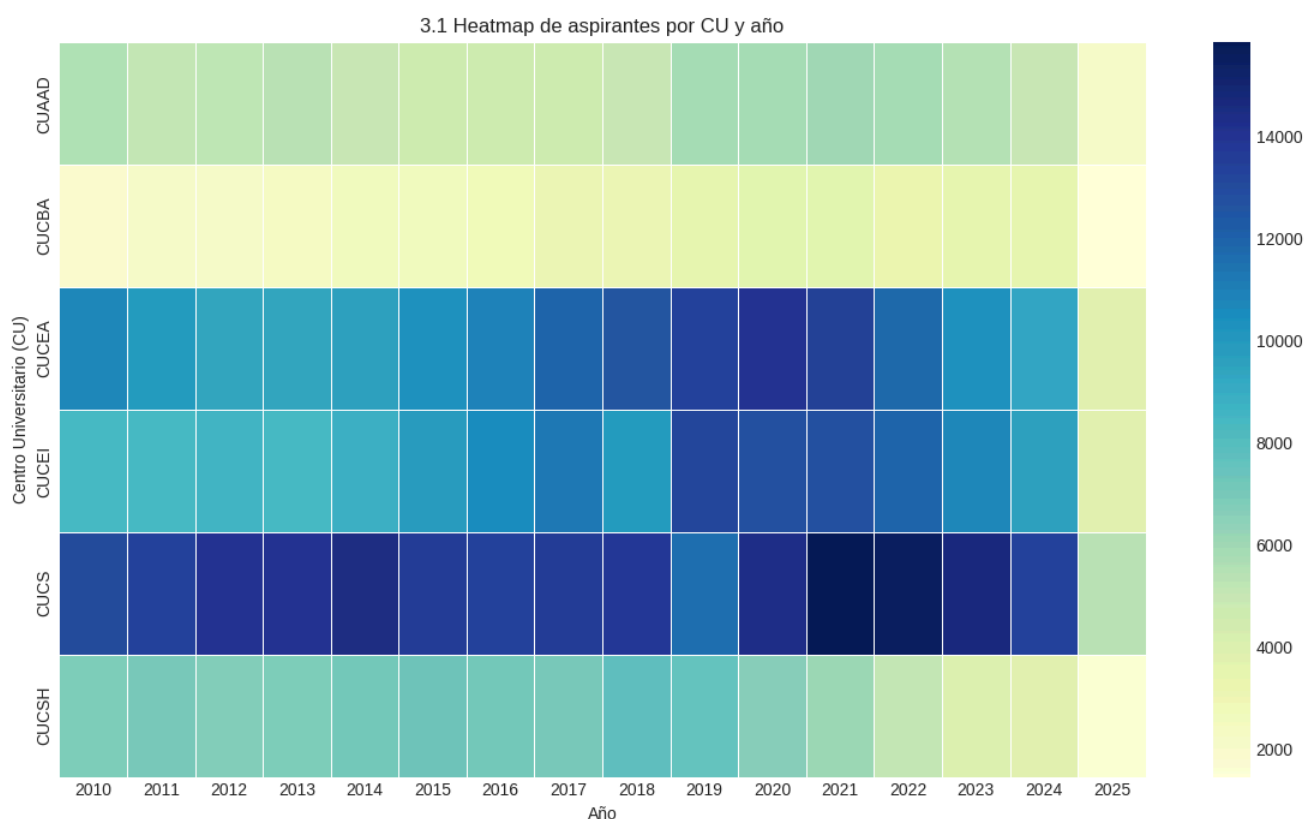
- CU
- AÑO
- ASPIRANTES

 **¿Qué revela?** Permite detectar periodos con alta o baja demanda por CU y si hay patrones estacionales o cambios fuertes.

Este heatmap muestra cuántos aspirantes ha tenido cada CU a lo largo de los años. Las tonalidades más oscuras indican mayor número de aspirantes.

```
# Agrupamos por CU y AÑO
cu_aspirantes = df_carreras.groupby(["CU", "AÑO"])["ASPIRANTES"].sum().unstack()

# Plot
plt.figure(figsize=(12, 7))
sns.heatmap(cu_aspirantes, cmap="YlGnBu", annot=False, linewidths=0.5)
plt.title("3.1 Heatmap de aspirantes por CU y año")
plt.xlabel("Año")
plt.ylabel("Centro Universitario (CU)")
plt.tight_layout()
plt.show()
```



### 3.2 Barras apiladas por admitidos por CU y carrera

🎯 **Objetivo:** Visualizar qué tantas carreras contribuyen a los admitidos totales por cada CU.

📊 **Tipo de gráfico:** Barras apiladas

📋 **Variables clave:**

- CU
- CARRERA
- ADMITIDOS

🔍 **¿Qué revela?** Permite ver si pocas carreras concentran la mayoría de los admitidos o si hay una distribución equilibrada en cada CU.

Las barras muestran cuántos admitidos aporta cada carrera dentro de su respectivo CU. Esto permite identificar carreras dominantes o centros con alta diversidad académica.

```
# Agrupamos admitidos por CU y carrera
admitidos_cu = df_carreras.groupby(["CU", "CARRERA"])["ADMITIDOS"].sum().reset_index()

# Tomamos los 6 CUs con más admitidos para visualización
top_cus = admitidos_cu.groupby("CU")["ADMITIDOS"].sum().nlargest(6).index
```

```
admitidos_top = admitidos_cu[admitidos_cu["CU"].isin(top_cus)]

# Pivot para barras apiladas
pivot = admitidos_top.pivot_table(index="CU", columns="CARRERA", values="ADMITIDOS", fill_value=0)

# Gráfico
pivot.plot(kind='bar', stacked=True, figsize=(14, 7), colormap='tab20')
plt.title("3.2 Admitidos por carrera y CU (Top 6 CUs)")
plt.ylabel("Número total de admitidos")
plt.xlabel("Centro Universitario (CU)")
plt.legend(loc='upper right', bbox_to_anchor=(1.2, 1.0), title="Carrera")
plt.tight_layout()
plt.show()
```



```
/tmp/ipython-input-2329049469.py:17: UserWarning: Tight layout not applied. The bottom and top margins cannot be made  
plt.tight_layout()
```

3.2 Admitidos por carrera y CU (Top 6 CUs)

SECCIÓN 4. Análisis por carrera

SECCIÓN 5. Impacto de la pandemia

5.1 Gráfica antes/durante/después de COVID (por carrera)

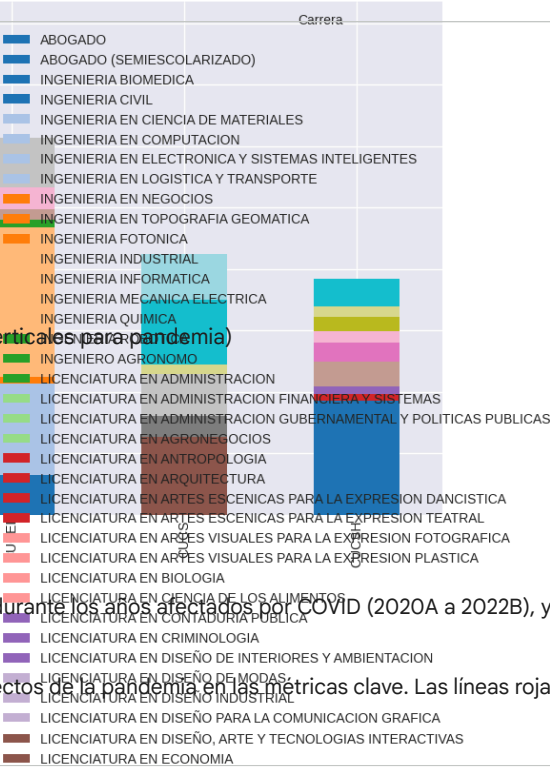
Tipo de gráfico: Líneas múltiples con marcadores de eventos (líneas verticales para pandemia)

Variables:

- PERIODO
- ASPIRANTES
- TASA\_DEMANDA
- PUNTAJE\_MINIMO
- CARRERA

¿Qué revela? Cambios bruscos en la demanda y dificultad de ingreso durante los años afectados por COVID (2020A a 2022B), y su comparación con los periodos anteriores y posteriores.

Se muestran las tendencias históricas de cada carrera destacando los efectos de la pandemia en las métricas clave. Las líneas rojas marcan el inicio (2020A) y final estimado (2023A) de la pandemia.



```

import matplotlib.pyplot as plt
import seaborn as sns

# Eventos importantes
eventos = {
    '2020A': 'Inicio COVID-19',
    '2021A': 'Fin pandemia'
}

def graficar_carrera_con_eventos(carrera):
    datos = df_carreras[df_carreras['CARRERA'] == carrera]

    fig, axs = plt.subplots(3, 1, figsize=(12, 10), sharex=True)

    # Aspirantes
    sns.lineplot(data=datos, x='PERIODO', y='ASPIRANTES', marker='o', ax=axs[0])
    axs[0].set_title(f'Aspirantes - {carrera}')
    axs[0].grid()

    # Tasa de Demanda
    sns.lineplot(data=datos, x='PERIODO', y='TASA_DEMANDA', marker='o', color='orange', ax=axs[1])
    axs[1].set_title('Tasa de Demanda')
    axs[1].grid()

    # PM
    sns.lineplot(data=datos, x='PERIODO', y='PUNTAJE_MINIMO', marker='o', color='green', ax=axs[2])
    axs[2].set_title('Puntaje Mínimo (PM)')
    axs[2].grid()

    # Líneas de eventos
    for ax in axs:
        for periodo, etiqueta in eventos.items():
            if periodo in datos['PERIODO'].values:
                ax.axvline(x=periodo, color='red', linestyle='--')
                ax.text(periodo, ax.get_ylim()[1]*0.95, etiqueta, rotation=90, color='red', verticalalignment='top')

    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

# Ejemplo de uso:
graficar_carrera_con_eventos('INGENIERIA EN COMPUTACION')

```



### 5.2 Comparación antes, durante y después de la pandemia

🎯 **Objetivo:** Comparar el número de aspirantes y el puntaje mínimo antes (pre-2020), durante (2020–2021) y después de la pandemia.

📊 **Tipo de gráfico:** Barras agrupadas o FacetGrid

📋 **Variables clave:**

- AÑO
- ASPIRANTES
- PUNTAJE\_MINIMO
- PERIODO\_PANDEMIA (etiquetado como "PRE", "DURANTE", "POST")

🔍 **¿Qué revela?** Si hubo una caída, estabilización o recuperación en la demanda o los puntajes después del impacto del COVID-19.

El gráfico compara métricas clave de ingreso antes, durante y después de la pandemia de COVID-19, para observar su impacto en la demanda y dificultad de acceso a las carreras.

```
# Clasificar periodo según pandemia
def clasificar_periodo(año):
    if año <= 2019:
        return "PRE"
```



```

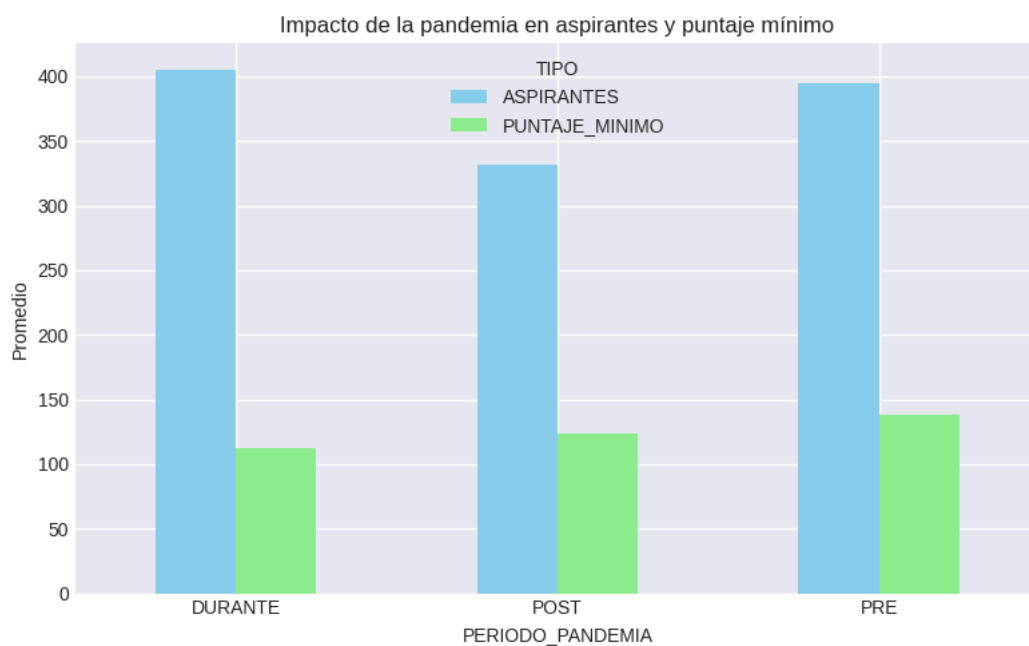
elif año in [2020, 2021]:
    return "DURANTE"
else:
    return "POST"

df_carreras["PERIODO_PANDEMIA"] = df_carreras["AÑO"].apply(clasificar_periodo)

# Agrupamos por periodo
df_pandemia = df_carreras.groupby("PERIODO_PANDEMIA")[["ASPIRANTES", "PUNTAJE_MINIMO"]].mean().reset_index()

# Gráfica
fig, ax1 = plt.subplots(figsize=(8, 5))
df_pandemia.plot(kind='bar', x="PERIODO_PANDEMIA", ax=ax1, color=["skyblue", "lightgreen"])
plt.title("Impacto de la pandemia en aspirantes y puntaje mínimo")
plt.ylabel("Promedio")
plt.xticks(rotation=0)
plt.grid(True)
plt.tight_layout()
plt.show()

```



### 5.3 Regresión lineal por periodo pandémico

🎯 **Objetivo general:** Evaluar con modelos de regresión lineal cómo se han comportado las variables clave (aspirantes, admitidos y puntaje mínimo) antes, durante y después de la pandemia, diferenciando por carrera y por centro universitario.

Las líneas de regresión ayudan a identificar tendencias claras de crecimiento o decrecimiento, incluso cuando los datos por sí solos no lo hacen evidente.

```

# 🧠 Clasificamos cada semestre como 'Antes', 'Durante' o 'Después' de la pandemia
def clasificar_periodo(p):
    año = int(p[:4])
    if año < 2020:
        return "Antes"
    elif 2020 <= año <= 2022:
        return "Durante"
    else:
        return "Después"

# Aplicamos clasificación al DataFrame
df_carreras["PERIODO_PANDEMIA"] = df_carreras["PERIODO"].apply(clasificar_periodo)

```

#### 5.3a Regresión por carrera (comparativa antes/durante/después)

📊 **Tipo de gráfico:**

- Implot() de Seaborn

- Una subgráfica por carrera
- Líneas de regresión separadas para los tres periodos pandémicos

#### Variables:

- AÑO
- ASPIRANTES
- ADMITIDOS
- PUNTAJE\_MINIMO
- PERIODO\_PANDEMIA
- CARRERA (filtrado por un conjunto clave)

#### ¿Qué revela?

- Pendiente positiva: crecimiento sostenido en la variable (ej. más aspirantes o mayor puntaje).
- Pendiente negativa: caída progresiva.
- Permite comparar si ciertas carreras se han vuelto más atractivas o han perdido interés, especialmente después del COVID.

Este gráfico muestra cómo evolucionaron los valores de aspirantes, admitidos y puntaje mínimo en carreras clave antes, durante y después de la pandemia. Las líneas de regresión ayudan a visualizar tendencias de recuperación, declive o estabilidad en el comportamiento de cada carrera.

```
from sklearn.linear_model import LinearRegression

carreras_clave = ["INGENIERIA BIOMEDICA"]
periodos = ["PRE", "DURANTE", "POST"]
variables = ["ASPIRANTES", "PUNTAJE_MINIMO", "ADMITIDOS"]

for carrera in carreras_clave:
    print(f"\n=== CARRERA: {carrera} ===")
    df_filtro = df_carreras[df_carreras["CARRERA"] == carrera]

    for variable in variables:
        print(f"\n--- Variable: {variable} ---")

        for periodo in periodos:
            sub = df_filtro[df_filtro["PERIODO_PANDEMIA"] == periodo].dropna(subset=[variable])
            if len(sub) >= 2:
                X = sub[["AÑO"]].values
                y = sub[variable].values
                model = LinearRegression().fit(X, y)
                r2 = model.score(X, y)
                pendiente = model.coef_[0]
                print(f"{periodo}: R² = {r2:.3f}, pendiente = {pendiente:.2f}")
            else:
                print(f"{periodo}: datos insuficientes")
```

```
=== CARRERA: INGENIERIA BIOMEDICA ===
```

```
--- Variable: ASPIRANTES ---
PRE: datos insuficientes
DURANTE: datos insuficientes
POST: datos insuficientes
```

```
--- Variable: PUNTAJE_MINIMO ---
PRE: datos insuficientes
DURANTE: datos insuficientes
POST: datos insuficientes
```

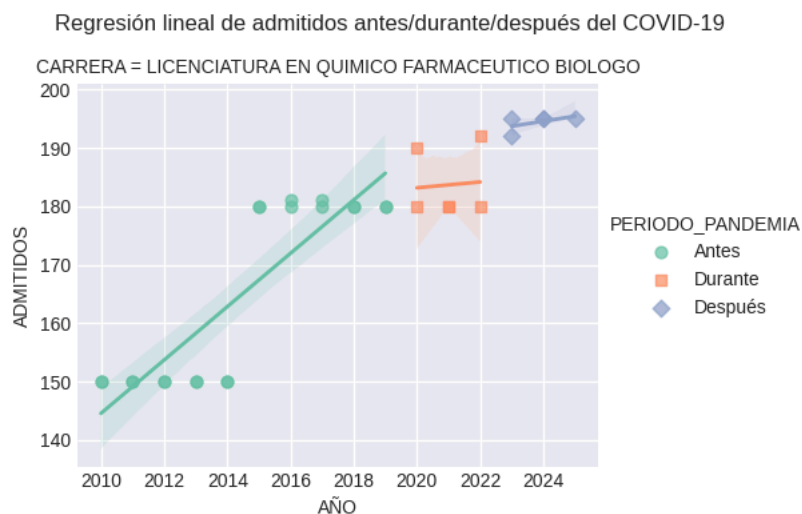
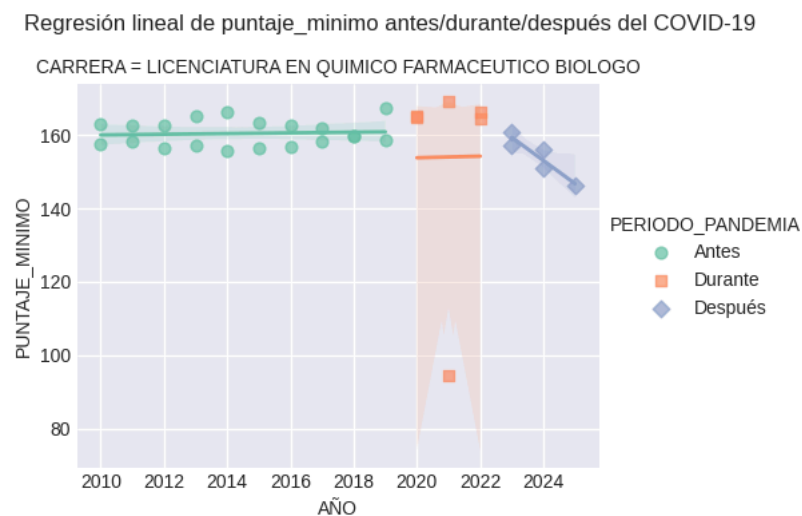
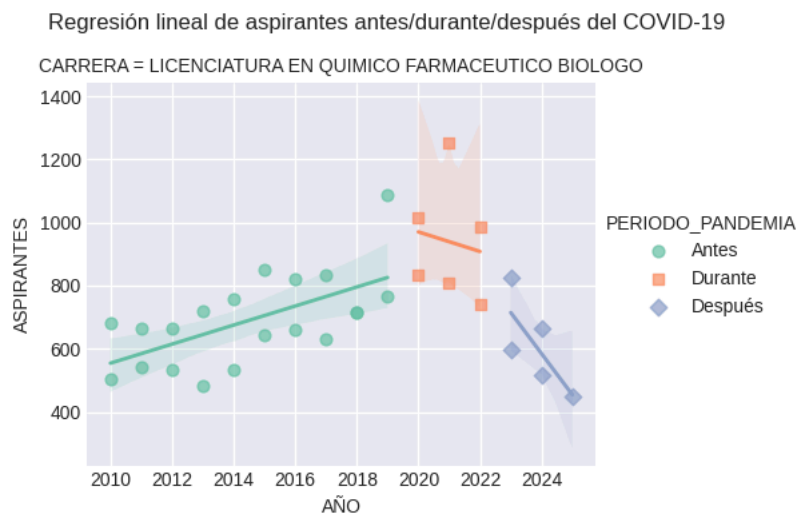
```
--- Variable: ADMITIDOS ---
PRE: datos insuficientes
DURANTE: datos insuficientes
POST: datos insuficientes
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# 🍷 Puedes modificar este diccionario para elegir tus carreras clave
carreras_clave = [
    "LICENCIATURA EN QUIMICO FARMACEUTICO BIOLOGO"
]
```

```
# 🔍 Visualizamos regresiones por cada variable en las carreras seleccionadas
```

```
for variable in ["ASPIRANTES", "PUNTAJE_MINIMO", "ADMITIDOS"]:  
    g = sns.lmplot(  
        data=df_carreras[df_carreras["CARRERA"].isin(carreras_clave)],  
        x="AÑO", y=variable,  
        col="CARRERA",  
        hue="PERIODO_PANDEMIA",  
        height=4, aspect=1.2,  
        markers=["o", "s", "D"],  
        palette="Set2",  
        scatter_kws={'s': 40, 'alpha': 0.7},  
        line_kws={'linewidth': 2}  
    )  
    g.fig.subplots_adjust(top=0.85)  
    g.fig.suptitle(f"Regresión lineal de {variable.lower()} antes/durante/después del COVID-19")  
  
    # Muestra la figura (esto va DESPUÉS de guardarla)  
    plt.show()
```



```
variable_labels = {
    "ASPIRANTES": "Applicants",
    "PUNTAJE_MINIMO": "Minimum Score",
    "ADMITIDOS": "Admitted"
}
```

```
import seaborn as sns
import matplotlib.pyplot as plt
from google.colab import files # Omitir esta línea si no usas Colab
```

```

# Traducir periodo pandémico
df_carreras["PANDEMIC_PERIOD"] = df_carreras["PERIODO_PANDEMIA"].map({
    "PRE": "Pre",
    "DURANTE": "During",
    "POST": "Post"
})

# Etiquetas en inglés para variables
variable_labels = {
    "ASPIRANTES": "Applicants",
    "PUNTAJE_MINIMO": "Minimum Score",
    "ADMITIDOS": "Admitted"
}

# Lista de carreras clave
key_careers = [
    "INGENIERIA EN COMPUTACION"
]

# Si deseas traducir nombres de carrera también (opcional)
career_translation = {
    "INGENIERIA EN COMPUTACION": "Computer Engineering"
}

# Aplicar traducción de carrera si lo deseas
df_carreras["CAREER"] = df_carreras["CARRERA"].replace(career_translation)

# Visualización y guardado
for variable in variable_labels:
    label = variable_labels[variable]

    g = sns.lmplot(
        data=df_carreras[df_carreras["CARRERA"].isin(key_careers)],
        x="AÑO", y=variable,
        col="CAREER",
        hue="PANDEMIC_PERIOD",
        height=4, aspect=1.2,
        markers=["o", "s", "D"],
        palette="Set2",
        scatter_kws={'s': 40, 'alpha': 0.7},
        line_kws={'linewidth': 2}
    )

    # Títulos
    g.set_ylabels(label)
    g.set_xlabels("Year")
    g.fig.subplots_adjust(top=0.85)
    g.fig.suptitle(f"Linear Regression of {label} Before, During, and After COVID-19")

    # Guardar archivo
    file_name = f"regression_{label.lower().replace(' ', '_')}.png"
    g.savefig(file_name, dpi=300, bbox_inches="tight")
    print(f"Figure '{file_name}' saved successfully.")

    # Descargar archivo (solo si estás en Colab)
    try:
        files.download(file_name)
        print(f"Downloading '{file_name}'...")
    except:
        pass

plt.show()

```

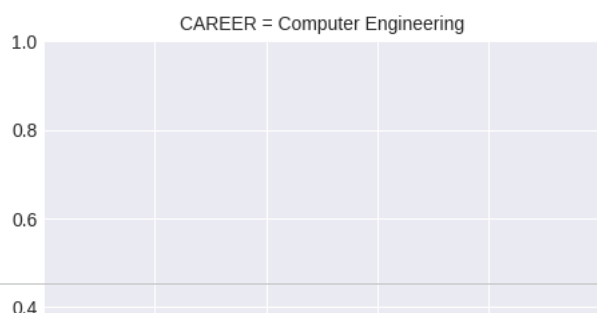
```

ValueError                                Traceback (most recent call last)
/tmp/ipython-input-2957615138.py in <cell line: 0>()
    34     label = variable_labels[variable]
    35
--> 36     g = sns.lmplot(
    37         data=df_carreras[df_carreras["CARRERA"].isin(key_careers)],
    38         x="AÑO", y=variable,

/usr/local/lib/python3.12/dist-packages/seaborn/regression.py in lmplot(data, x, y, hue, col, row, palette,
col_wrap, height, aspect, markers, sharex, sharey, hue_order, col_order, row_order, legend, legend_out, x_estimator,
x_bins, x_ci, scatter, fit_reg, ci, n_boot, units, seed, order, logistic, lowess, robust, logx, x_partial,
y_partial, truncate, x_jitter, y_jitter, scatter_kws, line_kws, facet_kws)
    629     markers = [markers] * n_markers
    630     if len(markers) != n_markers:
--> 631         raise ValueError("markers must be a singleton or a list of markers "
    632                             "for each level of the hue variable")
    633     facets.hue_kws = {"marker": markers}

```

**ValueError:** markers must be a singleton or a list of markers for each level of the hue variable



### 5.3b Regresión por centro universitario (promedio anual)

#### 📌 Tipo de gráfico:

- lmplot() por CU (una subgráfica por centro)
- Cada punto es un promedio anual de todos los programas del CU
- Colores según el periodo pandémico

#### 📌 Variables:

Next steps:

[Explain error](#)

- CU
- AÑO
- ASPIRANTES (promedio)
- ADMITIDOS (promedio)
- PUNTAJE\_MINIMO (promedio)
- PERIODO\_PANDEMIA

#### 🔍 ¿Qué revela?

- Identifica CUs que perdieron o ganaron demanda tras la pandemia.
- Visualiza si la competitividad cambió (por puntaje mínimo promedio).
- Puede destacar desigualdad en la recuperación entre centros regionales y metropolitanos.

Esta visualización compara las tendencias de ingreso y competitividad por centro universitario de la UDG antes, durante y después del COVID-19. Las líneas de regresión permiten analizar si el interés por cada centro ha ido en aumento o ha disminuido, así como el impacto en la dificultad de ingreso.

```

from sklearn.linear_model import LinearRegression

# Agrupamos por CU, periodo y año (esto ya lo hiciste)
df_cu_agg = df_carreras.groupby(['CU', 'PERIODO_PANDEMIA', 'AÑO'])[
    ['ASPIRANTES', 'ADMITIDOS', 'PUNTAJE_MINIMO']
].mean().reset_index()

# Variables y periodos a analizar
variables = ['ASPIRANTES', 'ADMITIDOS', 'PUNTAJE_MINIMO']
periodos = ['PRE', 'DURANTE', 'POST']
centros = df_cu_agg['CU'].unique()

# Cálculo de regresiones
for var in variables:
    print(f"\n=== VARIABLE: {var} ===")
    for cu in centros:

```

```

for cu in centros:
    for periodo in periodos:
        sub = df_cu_agg[
            (df_cu_agg["CU"] == cu) &
            (df_cu_agg["PERIODO_PANDEMIA"] == periodo)
        ].dropna(subset=[var])
        if len(sub) >= 2:
            X = sub[["AÑO"]].values
            y = sub[var].values
            model = LinearRegression().fit(X, y)
            r2 = model.score(X, y)
            pendiente = model.coef_[0]
            print(f"{cu} | {periodo}: R² = {r2:.3f}, pendiente = {pendiente:.2f}")
        else:
            print(f"{cu} | {periodo}: datos insuficientes")

CUAAD | PRE: datos insuficientes
CUAAD | DURANTE: datos insuficientes
CUAAD | POST: datos insuficientes
CUCBA | PRE: datos insuficientes
CUCBA | DURANTE: datos insuficientes
CUCBA | POST: datos insuficientes
CUCEA | PRE: datos insuficientes
CUCEA | DURANTE: datos insuficientes
CUCEA | POST: datos insuficientes
CUCEI | PRE: datos insuficientes
CUCEI | DURANTE: datos insuficientes
CUCEI | POST: datos insuficientes
CUCS | PRE: datos insuficientes
CUCS | DURANTE: datos insuficientes
CUCS | POST: datos insuficientes
CUCSH | PRE: datos insuficientes
CUCSH | DURANTE: datos insuficientes
CUCSH | POST: datos insuficientes

=== VARIABLE: ADMITIDOS ===
CUAAD | PRE: datos insuficientes
CUAAD | DURANTE: datos insuficientes
CUAAD | POST: datos insuficientes
CUCBA | PRE: datos insuficientes
CUCBA | DURANTE: datos insuficientes
CUCBA | POST: datos insuficientes
CUCEA | PRE: datos insuficientes
CUCEA | DURANTE: datos insuficientes
CUCEA | POST: datos insuficientes
CUCEI | PRE: datos insuficientes
CUCEI | DURANTE: datos insuficientes
CUCEI | POST: datos insuficientes
CUCS | PRE: datos insuficientes
CUCS | DURANTE: datos insuficientes
CUCS | POST: datos insuficientes
CUCSH | PRE: datos insuficientes
CUCSH | DURANTE: datos insuficientes
CUCSH | POST: datos insuficientes

=== VARIABLE: PUNTAJE_MINIMO ===
CUAAD | PRE: datos insuficientes
CUAAD | DURANTE: datos insuficientes
CUAAD | POST: datos insuficientes
CUCBA | PRE: datos insuficientes
CUCBA | DURANTE: datos insuficientes
CUCBA | POST: datos insuficientes
CUCEA | PRE: datos insuficientes
CUCEA | DURANTE: datos insuficientes
CUCEA | POST: datos insuficientes
CUCEI | PRE: datos insuficientes
CUCEI | DURANTE: datos insuficientes
CUCEI | POST: datos insuficientes
CUCS | PRE: datos insuficientes
CUCS | DURANTE: datos insuficientes
CUCS | POST: datos insuficientes
CUCSH | PRE: datos insuficientes
CUCSH | DURANTE: datos insuficientes
CUCSH | POST: datos insuficientes

```

```

# Agrupar por CU, PERIODO_PANDEMIA y AÑO
df_cu_agg = df_carreras.groupby(['CU', 'PERIODO_PANDEMIA', 'AÑO'])[
    ['ASPIRANTES', 'ADMITIDOS', 'PUNTAJE_MINIMO']
].mean().reset_index()

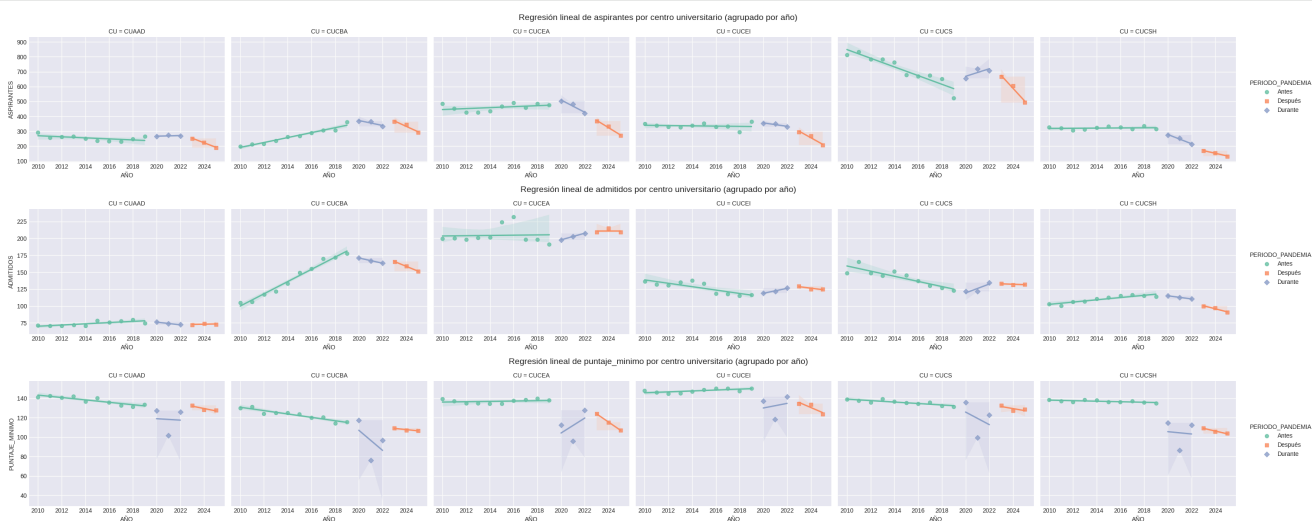
# Crear gráficos de regresión por CU
import seaborn as sns

```

```
import matplotlib.pyplot as plt

# Lista de variables a graficar
variables = ['ASPIRANTES', 'ADMITIDOS', 'PUNTAJE_MINIMO']

for var in variables:
    g = sns.lmplot(
        data=df_cu_agg,
        x="AÑO", y=var,
        col="CU",
        hue="PERIODO_PANDEMIA",
        height=4, aspect=1.2,
        markers=["o", "s", "D"],
        palette="Set2"
    )
    g.fig.subplots_adjust(top=0.85)
    g.fig.suptitle(f"Regresión lineal de {var.lower()} por centro universitario (agrupado por año)", fontsize=14)
    plt.show()
```



```
import seaborn as sns
import matplotlib.pyplot as plt
from google.colab import files # Elimina si no estás en Google Colab

# Traducir nombre de columna y valores
df_carreras["PANDEMIC_PERIOD"] = df_carreras["PERIODO_PANDEMIA"].map({
    "PRE": "Pre",
    "DURANTE": "During",
    "POST": "Post"
})

# Agrupar por CU, PANDEMIC_PERIOD y AÑO
df_cu_agg = df_carreras.groupby(['CU', 'PANDEMIC_PERIOD', 'AÑO'])[
    ['ASPIRANTES', 'ADMITIDOS', 'PUNTAJE_MINIMO']
].mean().reset_index()

# Etiquetas en inglés para los títulos
variable_labels = {
    "ASPIRANTES": "Applicants",
    "ADMITIDOS": "Admitted",
    "PUNTAJE_MINIMO": "Minimum Score"
}

# Lista para nombres de archivo
saved_files = []
```



```
# Crear, guardar y descargar los gráficos
for var, label in variable_labels.items():
    g = sns.lmplot(
        data=df_cu_agg,
        x="AÑO", y=var,
        col="CU",
        hue="PANDEMIC_PERIOD",
        height=4, aspect=1.2,
        markers=["o", "s", "D"],
        palette="Set2"
    )
    g.set_ylabels(label)
    g.set_xlabels("Year")
    g.fig.subplots_adjust(top=0.85)
    g.fig.suptitle(f"Linear Regression of {label} by University Center (Grouped by Year)", fontsize=14)

    file_name = f"regression_{label.lower().replace(' ', '_')}_by_cu.png"
    g.savefig(file_name, dpi=300, bbox_inches="tight")
    print(f"Figure '{file_name}' saved successfully.")
    saved_files.append(file_name)

plt.show()

# Descargar los archivos (solo en Colab)
for file in saved_files:
    try:
        files.download(file)
        print(f"Downloading '{file}'...")
    except Exception as e:
        print(f"Error downloading '{file}': {e}")
```

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-2042184069.py in <cell line: 0>()
    27 # Crear, guardar y descargar los gráficos
    28 for var, label in variable_labels.items():
--> 29     g = sns.lmplot(
    30         data=df_cu_agg,
    31         x="AÑO", y=var,

----- 5 frames -----
/usr/local/lib/python3.12/dist-packages/matplotlib/gridspec.py in __init__(self, nrows, ncols, height_ratios, width_ratios)
    49         f"Number of rows must be a positive integer, not {nrows!r}")
    50     if not isinstance(ncols, Integral) or ncols <= 0:
--> 51         raise ValueError(
    52             f"Number of columns must be a positive integer, not {ncols!r}")
    53     self._nrows, self._ncols = nrows, ncols

ValueError: Number of columns must be a positive integer, not 0

<Figure size 0x400 with 0 Axes>
```

Next steps: [Explain error](#)

#### 5.4 Impacto comparativo de la pandemia (DURANTE → POST)

🎯 **Objetivo general:** Medir y visualizar qué carreras y centros universitarios (CU) fueron más afectados o beneficiados después del periodo más crítico de la pandemia de COVID-19, comparando directamente el cambio entre los promedios de los semestres DURANTE y POST-pandemia.

```
# Aseguramos que el año sea numérico
df_carreras['AÑO'] = df_carreras['AÑO'].astype(int)

# Clasificación robusta por periodo
def clasificar_periodo(año):
    if año <= 2019:
        return "PRE"
    elif 2020 <= año <= 2022:
        return "DURANTE"
    else:
        return "POST"

# Aplicamos la clasificación
```

```
df_carreras['PERIODO_PANDEMIA'] = df_carreras['AÑO'].apply(clasificar_periodo)

# Confirmamos
print("Periodos clasificados:", df_carreras['PERIODO_PANDEMIA'].unique())
```

#### 5.4a. Carreras más afectadas tras la pandemia

 Tipo de gráfico: Barras horizontales apiladas (por variable), ordenadas por cambio total en la carrera.

 Variables analizadas:

- ASPIRANTES
- ADMITIDOS
- PUNTAJE\_MINIMO

 ¿Cómo se calculó? Para cada carrera:

1. Se calculó el promedio de cada variable durante los periodos DURANTE (2020–2022) y POST (2023 en adelante).
2. Luego se restó:

POST – DURANTE

Esto da el cambio absoluto promedio en el número de aspirantes, admitidos y el puntaje mínimo.

3. Se sumaron los tres valores para obtener un indicador compuesto de cambio total.

 ¿Qué revela? Las 15 carreras con mayor caída en aspirantes, admitidos y/o puntaje mínimo, lo que sugiere una pérdida de interés o aumento de dificultad tras la pandemia.

Este gráfico muestra las 15 carreras más afectadas tras el periodo pandémico, con base en la disminución acumulada de aspirantes, admitidos y puntaje mínimo. Refleja posibles cambios de percepción o accesibilidad hacia dichas carreras.

```
# Aseguramos importación de numpy
import numpy as np

# Agrupamos y deshacemos jerarquía por PERIODO_PANDEMIA
df_variacion_carreras = df_carreras.groupby(["CARRERA", "PERIODO_PANDEMIA"])[
    ["ASPIRANTES", "ADMITIDOS", "PUNTAJE_MINIMO"]
].mean().unstack(level=-1)

# Aseguramos que las columnas esperadas existan
for periodo in ["DURANTE", "POST"]:
    for var in ["ASPIRANTES", "ADMITIDOS", "PUNTAJE_MINIMO"]:
        if (var, periodo) not in df_variacion_carreras.columns:
            df_variacion_carreras[(var, periodo)] = np.nan

# Calcular diferencias POST – DURANTE
diferencias_carreras = pd.DataFrame({
    "ASPIRANTES": df_variacion_carreras[("ASPIRANTES", "POST")] - df_variacion_carreras[("ASPIRANTES", "DURANTE")],
    "ADMITIDOS": df_variacion_carreras[("ADMITIDOS", "POST")] - df_variacion_carreras[("ADMITIDOS", "DURANTE")],
    "PUNTAJE_MINIMO": df_variacion_carreras[("PUNTAJE_MINIMO", "POST")] - df_variacion_carreras[("PUNTAJE_MINIMO", "DURANTE")],
})
diferencias_carreras["TOTAL_CAMBIO"] = diferencias_carreras.sum(axis=1)

# Top 15 más afectadas
top_15_caída = diferencias_carreras.sort_values("TOTAL_CAMBIO").head(15)

# Gráfica
top_15_caída[["ASPIRANTES", "ADMITIDOS", "PUNTAJE_MINIMO"]].plot(
    kind="barh", figsize=(10, 7), color=["skyblue", "lightgreen", "salmon"]
)
plt.title("5.4a Carreras más afectadas tras la pandemia (cambio DURANTE → POST)")
plt.xlabel("Cambio promedio")
plt.grid(True)
plt.tight_layout()
plt.show()
```

#### ◆ 5.4b. Centros universitarios más afectados tras la pandemia

 Tipo de gráfico: Barras horizontales apiladas, una por CU, con colores diferenciados por variable.

 Variables analizadas:

- ASPIRANTES
- ADMITIDOS
- PUNTAJE\_MINIMO

📊 ¿Cómo se calculó? Para cada CU:

1. Se obtuvo el promedio DURANTE y POST-pandemia para cada variable.
2. Se calculó la diferencia: POST – DURANTE.
3. Luego, se sumó el cambio de las tres variables para obtener una medida global del impacto.

🔍 ¿Qué revela? Permite identificar qué centros universitarios tuvieron un retroceso significativo tras la pandemia, y en qué medida fueron afectados en términos de demanda y nivel de ingreso.

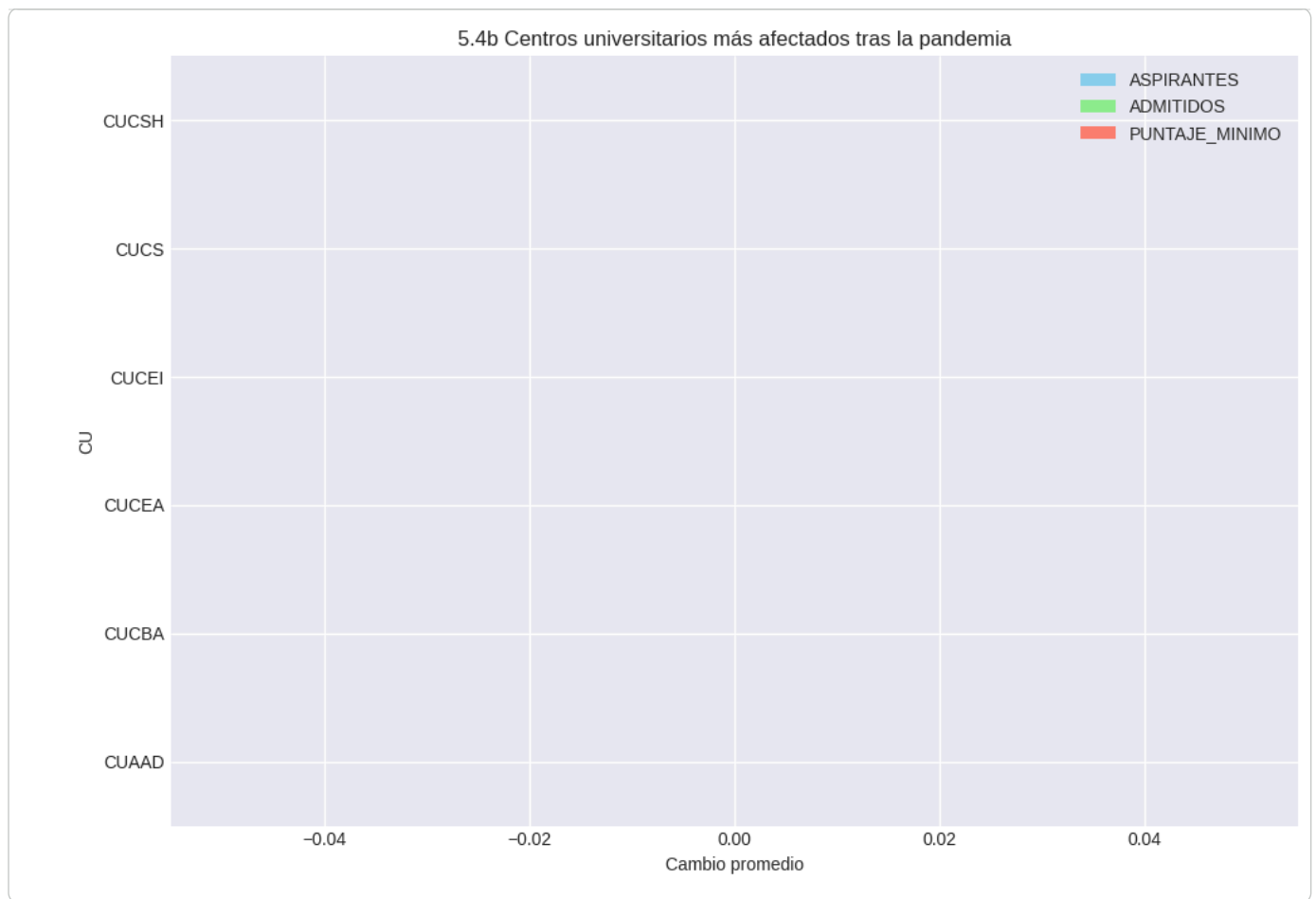
Este análisis muestra los centros universitarios que presentaron mayor disminución en aspirantes, admitidos y puntaje mínimo tras el periodo pandémico, evidenciando posibles efectos en su oferta educativa o percepción institucional.

```
# Agrupamos por CU y PERIODO_PANDEMIA
df_variacion_cu = df_carreras.groupby(["CU", "PERIODO_PANDEMIA"])[
    ["ASPIRANTES", "ADMITIDOS", "PUNTAJE_MINIMO"]
].mean().unstack(level=-1)

# Aseguramos que existan
for periodo in ["DURANTE", "POST"]:
    for var in ["ASPIRANTES", "ADMITIDOS", "PUNTAJE_MINIMO"]:
        if (var, periodo) not in df_variacion_cu.columns:
            df_variacion_cu[(var, periodo)] = np.nan

# Calcular diferencias
diferencias_cu = pd.DataFrame({
    "ASPIRANTES": df_variacion_cu[("ASPIRANTES", "POST")] - df_variacion_cu[("ASPIRANTES", "DURANTE")],
    "ADMITIDOS": df_variacion_cu[("ADMITIDOS", "POST")] - df_variacion_cu[("ADMITIDOS", "DURANTE")],
    "PUNTAJE_MINIMO": df_variacion_cu[("PUNTAJE_MINIMO", "POST")] - df_variacion_cu[("PUNTAJE_MINIMO", "DURANTE")]
})
diferencias_cu["TOTAL_CAMBIO"] = diferencias_cu.sum(axis=1)

# Ordenar y graficar
diferencias_cu.sort_values("TOTAL_CAMBIO")[["ASPIRANTES", "ADMITIDOS", "PUNTAJE_MINIMO"]].plot(
    kind="barh", figsize=(10, 7), color=["skyblue", "lightgreen", "salmon"]
)
plt.title("5.4b Centros universitarios más afectados tras la pandemia")
plt.xlabel("Cambio promedio")
plt.grid(True)
plt.tight_layout()
plt.show()
```



## > SECCIÓN 6. Distribuciones y correlaciones

↳ 5 cells hidden

## ✓ SECCIÓN 7. Segmentación y Clusters

🎯 **Objetivo:** Agrupar las carreras de acuerdo a su comportamiento en variables clave de ingreso como aspirantes, admitidos, tasa de demanda y puntaje mínimo, con el fin de identificar perfiles comunes de carrera:

- Altamente competitivas
- Intermedias
- Baja demanda o accesibilidad

📋 **Variables utilizadas:**

- ASPIRANTES
- ADMITIDOS
- TASA\_DEMANDA (calculada como ASPIRANTES / ADMITIDOS)
- PUNTAJE\_MINIMO

⚙️ **Metodología:**

1. Estandarización de variables: Se normalizan para evitar que una variable domine por su escala.
2. Reducción de dimensionalidad con PCA:
  - PCA1 representa la demanda general (aspirantes/admitidos).
  - PCA2 representa la selectividad o dificultad de ingreso (puntaje mínimo).
3. Agrupamiento (KMeans): Se generan 3 grupos con base en las variables estandarizadas.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import seaborn as sns

# Asegurar cálculo de TASA_DEMANDA
if 'TASA_DEMANDA' not in df_carreras.columns:
    df_carreras["TASA_DEMANDA"] = df_carreras["ASPIRANTES"] / df_carreras["ADMITIDOS"]

# Promedios por carrera
df_cluster = df_carreras.groupby("CARRERA")[
    ["ASPIRANTES", "ADMITIDOS", "TASA_DEMANDA", "PUNTAJE_MINIMO"]
].mean()

# Escalado de características
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_cluster)

# Reducción de dimensiones con PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
explained_var = pca.explained_variance_ratio_ * 100 # porcentaje

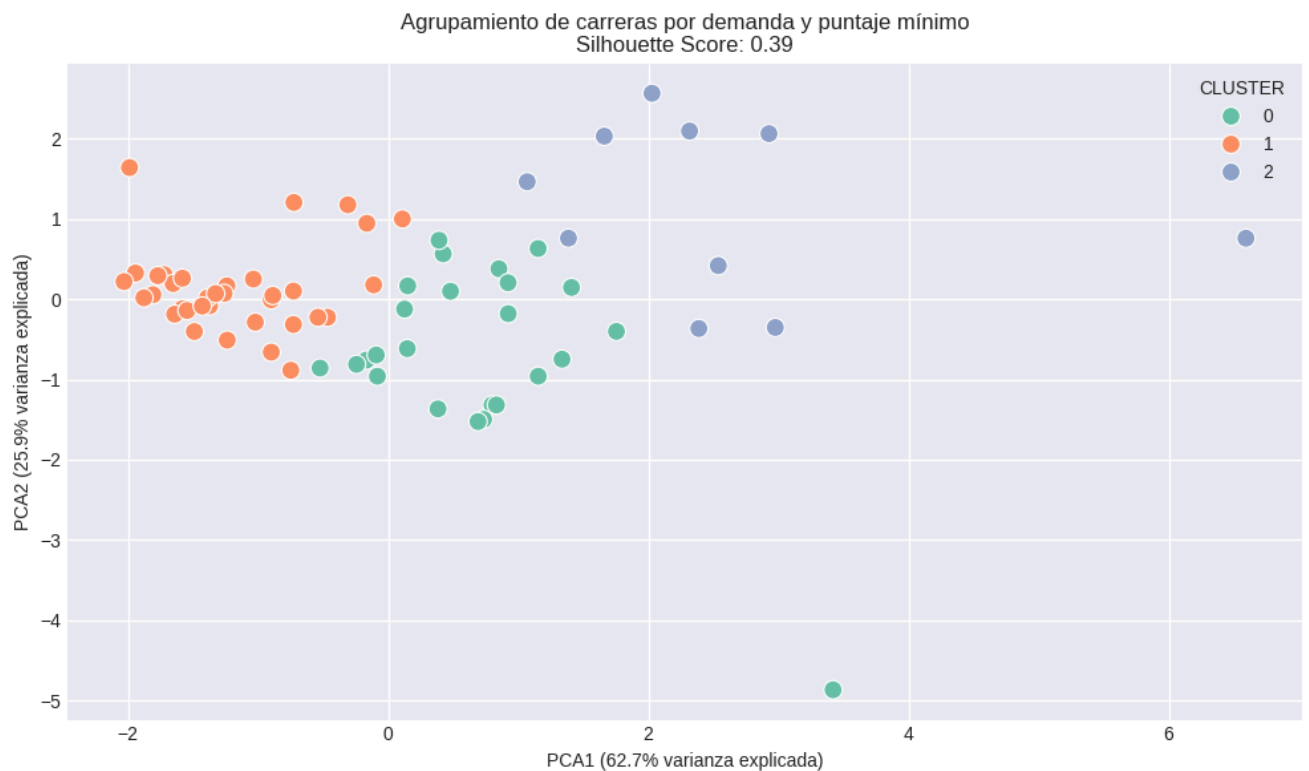
# K-Means clustering
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

# Cálculo del Silhouette Score
sil_score = silhouette_score(X_scaled, clusters)

# DataFrame PCA con clusters
df_cluster_pca = pd.DataFrame(X_pca, columns=["PCA1", "PCA2"], index=df_cluster.index)
df_cluster_pca["CLUSTER"] = clusters

# Gráfica con etiquetas mejoradas
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df_cluster_pca,
    x="PCA1", y="PCA2", hue="CLUSTER", palette="Set2", s=100
)
plt.title(f"Agrupamiento de carreras por demanda y puntaje mínimo\nSilhouette Score: {sil_score:.2f}")
plt.xlabel(f"PCA1 ({explained_var[0]:.1f}% varianza explicada)")
plt.ylabel(f"PCA2 ({explained_var[1]:.1f}% varianza explicada)")
plt.grid(True)
plt.tight_layout()
plt.show()

```



Interpretación de los Clusters 📌 Cluster 0:

- Carreras con menor número de aspirantes y puntajes bajos
- Accesibles o de menor demanda

📌 Cluster 1:

- Carreras de demanda y dificultad intermedia
- Perfil mixto

📌 Cluster 2:

- Carreras muy solicitadas y con puntajes mínimos altos
- Alta selectividad, como medicina o ingenierías populares

```
import pandas as pd

# Guardamos resultados
df_resultados_cluster = df_cluster.copy()
df_resultados_cluster[["PCA1", "PCA2", "CLUSTER"]] = df_cluster_pca[["PCA1", "PCA2", "CLUSTER"]]

# Mostramos tabla ordenada
df_resultados_cluster_sorted = df_resultados_cluster.sort_values("CLUSTER")
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from IPython.display import display

display(df_resultados_cluster_sorted)
```

|         | TIPO                                             | ASPIRANTES  | ADMITIDOS  | TASA_DEMANDA | PUNTAJE_MINIMO | PCA1     | PCA2      | CLUSTER |
|---------|--------------------------------------------------|-------------|------------|--------------|----------------|----------|-----------|---------|
| CARRERA |                                                  |             |            |              |                |          |           |         |
|         | ABOGADO (SEMIESCOLARIZADO)                       | 489.903226  | 193.935484 | 2.553702     | 130.682123     | 0.421958 | 0.564412  | 0       |
|         | INGENIERIA BIOMEDICA                             | 263.064516  | 77.354839  | 3.485404     | 166.202910     | 0.797460 | -1.317426 | 0       |
|         | INGENIERIA CIVIL                                 | 570.935484  | 133.483871 | 4.279240     | 155.886145     | 1.335592 | -0.748564 | 0       |
|         | INGENIERIA EN COMPUTACION                        | 598.806452  | 237.645161 | 2.617576     | 144.771626     | 1.152020 | 0.630371  | 0       |
|         | INGENIERIA INFORMATICA                           | 415.483871  | 213.903226 | 1.908120     | 135.124226     | 0.389898 | 0.733315  | 0       |
|         | ...                                              | ...         | ...        | ...          | ...            | ...      | ...       | ...     |
|         | LICENCIATURA EN MEDICINA VETERINARIA Y ZOOTECNIA | 791.354839  | 243.064516 | 3.220707     | 138.363726     | 1.385186 | 0.759032  | 2       |
|         | LICENCIATURA EN PSICOLOGIA                       | 1243.741935 | 204.580645 | 6.065356     | 158.054523     | 2.975470 | -0.353264 | 2       |
|         | LICENCIATURA EN ENFERMERIA                       | 1190.032258 | 248.741935 | 5.091059     | 145.458858     | 2.537132 | 0.417089  | 2       |
|         | LICENCIATURA EN ADMINISTRACION                   | 833.483871  | 375.548387 | 2.245781     | 133.355642     | 1.660509 | 2.030112  | 2       |
|         | MEDICO CIRUJANO Y PARTERO                        | 2897.967742 | 339.258065 | 8.320116     | 171.798223     | 6.593542 | 0.758594  | 2       |

71 rows x 7 columns

Next steps: [Generate code with df\\_resultados\\_cluster\\_sorted](#) [New interactive sheet](#)

>  Sección 8. Detección de Outliers

↳ 3 cells hidden

✓ PASO 3. Modelos Predictivos y Perfiles de Carrera

✓ Predicción Interactiva de Aspirantes y Admitidos con Series de Tiempo (Prophet)

 **Objetivo:** Estimar la cantidad futura de aspirantes y admitidos por carrera y centro universitario, a partir de datos históricos y modelos de series de tiempo.

Modelo utilizado: Facebook Prophet (Series de tiempo con componentes de tendencia, estacionalidad y eventos).

Variables de entrada:

- Fecha (ds generada a partir del PERIODO)
- CU
- CARRERA
- ASPIRANTES
- ADMITIDOS

Variables a predecir:

- ASPIRANTES
- ADMITIDOS

Interfaz: Menús interactivos permiten seleccionar CU, carrera y número de semestres a predecir.

Tendencias futuras por programa académico y centro, útiles para la planeación de admisiones, promoción universitaria, y asignación de recursos institucionales.

```
#  
# 0. Crear columna 'ds' a partir de PERIODO  
#  
# (ejecuta esto una sola vez, antes de usar Prophet o groupby("ds"))  
  
def periodo_to_date(periodo):  
    """  
    Convierte un string 'YYYYA' o 'YYYYB' en la fecha del inicio  
    del semestre correspondiente (A = 1-ene, B = 1-jul).  
    """  
    try:
```

```

        year = int(periodo[:4])
        ciclo = periodo[4].upper()
        return pd.to_datetime(f"{year}-01-01") if ciclo == "A" else pd.to_datetime(f"{year}-07-01")
    except Exception:
        return pd.NaT # Devuelve NaT si el formato no es válido

# Generar la nueva columna
df_carreras["ds"] = df_carreras["PERIODO0"].apply(periodo_to_date)

# Eliminar filas que no se pudieron convertir
df_carreras = df_carreras.dropna(subset=["ds"])

```

```

# 
# 1. Imports y configuración general
# 
import pandas as pd
import matplotlib.pyplot as plt
from prophet import Prophet
import ipywidgets as widgets
from ipywidgets import interactive, Dropdown, FloatSlider
from IPython.display import display, clear_output

```

Aquí se definen dos funciones principales:

***train\_prophet\_model(ts\_df)***: Recibe un DataFrame con dos columnas obligatorias (ds como fecha y y como variable a predecir) y entrena un modelo Prophet sobre esos datos. El modelo identifica automáticamente patrones de tendencia, estacionalidad y posibles rupturas estructurales.

***predict\_prophet(model, periods)***: Toma un modelo Prophet ya entrenado y genera predicciones a futuro, para el número de semestres que el usuario especifique. Utiliza una frecuencia de 6 meses (un semestre académico) y devuelve un DataFrame con fechas, valores pronosticados y sus intervalos de confianza.

Estas funciones encapsulan la lógica del modelo para que puedan ser reutilizadas fácilmente en distintos escenarios (por CU, por carrera o por UDG total).

```

# 
# 2. Funciones auxiliares: Prophet
# 
def train_prophet_model(ts_df):
    """
    Entrena un modelo Prophet con columnas ['ds','y'].
    """
    model = Prophet()
    model.fit(ts_df)
    return model

def predict_prophet(model, periods):
    """
    Devuelve pronóstico a 'periods' semestres a futuro.
    """
    future = model.make_future_dataframe(periods=periods, freq="6MS") # 6 meses ≈ 1 semestre
    forecast = model.predict(future)
    return forecast[["ds", "yhat", "yhat_lower", "yhat_upper"]]

```

En esta parte, se construye una visión general del comportamiento histórico de aspirantes y admitidos en toda la Universidad:

1. Se agrupan los datos por fecha (ds) y se suman los valores de aspirantes y admitidos.
2. Se renombra cada variable como y, para cumplir con el formato requerido por Prophet.
3. Se entrenan dos modelos Prophet: uno para aspirantes y otro para admitidos.
4. Finalmente, se generan predicciones para los próximos 4 semestres (2 años).

Esta sección proporciona una base de referencia global que puede usarse cuando no se desea filtrar por centro o carrera.



```

#
# 3. Modelos y pronósticos UDG total
#
df_udg_ts = (
    df_carreras
    .groupby("ds")[["ASPIRANTES", "ADMITIDOS"]]
    .sum()
    .reset_index()
)

# DataFrames para Prophet
df_udg_asp = df_udg_ts.rename(columns={"ASPIRANTES": "y"})[["ds", "y"]]
df_udg_adm = df_udg_ts.rename(columns={"ADMITIDOS": "y"})[["ds", "y"]]

model_udg_asp = train_prophet_model(df_udg_asp)
model_udg_adm = train_prophet_model(df_udg_adm)

# Pronóstico inicial: 4 semestres (≈ 2 años)
forecast_udg_asp = predict_prophet(model_udg_asp, periods=4)
forecast_udg_adm = predict_prophet(model_udg_adm, periods=4)

```

```

-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-2758703050.py in <cell line: 0>()
    13 df_udg_adm = df_udg_ts.rename(columns={"ADMITIDOS": "y"})[["ds", "y"]]
    14
--> 15 model_udg_asp = train_prophet_model(df_udg_asp)
    16 model_udg_adm = train_prophet_model(df_udg_adm)
    17

-----
2 frames -----
/usr/local/lib/python3.12/dist-packages/prophet/forecaster.py in _load_stan_backend(self, stan_backend)
    169         self.stan_backend = StanBackendEnum.get_backend_class(stan_backend)()
    170
--> 171         logger.debug("Loaded stan backend: %s", self.stan_backend.get_type())
    172
    173     def validate_inputs(self):

AttributeError: 'Prophet' object has no attribute 'stan_backend'

```

Next steps: [Explain error](#)

Esta función (plot\_predictions) permite generar predicciones ajustadas según los filtros que seleccione el usuario:

- Si no se selecciona ningún filtro (Todos los Centros, Todas las Carreras), se muestran las predicciones globales del Paso 3.
- Si se elige un CU o una carrera, se filtran los datos correspondientes, se agrupan por fecha y se entrena un nuevo modelo Prophet sobre esos datos específicos.
- En caso de no existir información histórica suficiente, se muestra un mensaje de advertencia.
- Si hay datos disponibles, se muestra la tabla de predicciones para aspirantes y admitidos, y se grafican las curvas generadas por el modelo.

Esta sección permite analizar con precisión la evolución de la demanda académica y sugiere posibles escenarios futuros.

```

import matplotlib.pyplot as plt
from ipywidgets import Dropdown, FloatSlider, interactive
from IPython.display import display, clear_output

cu_options = ["Todos los Centros"] + sorted(df_carreras["CU"].dropna().unique())
carrera_options = ["Todas las Carreras"] + sorted(df_carreras["CARRERA"].dropna().unique())

cu_dropdown = Dropdown(options=cu_options, description="Centro:")
carrera_dropdown = Dropdown(options=carrera_options, description="Carrera:")
periods_slider = FloatSlider(min=1, max=10, step=1, value=4, description="Semestres:")

def plot_predictions(selected_cu, selected_carrera, periods_to_predict):
    clear_output(wait=True)
    display(cu_dropdown, carrera_dropdown, periods_slider)

    if selected_cu == "Todos los Centros" and selected_carrera == "Todas las Carreras":

```

```

print(f"🧠 UDG Total – Próximos {int(periods_to_predict)} semestres")
display(forecast_udg_asp.head(int(periods_to_predict)))
display(forecast_udg_adm.head(int(periods_to_predict)))
model_udg_asp.plot(forecast_udg_asp); plt.title("Aspirantes UDG"); plt.show()
model_udg_adm.plot(forecast_udg_adm); plt.title("Admitidos UDG"); plt.show()
return

if selected_carrera == "Todas las Carreras":
    filtro = df_carreras["CU"] == selected_cu
    etiqueta = f"CU: {selected_cu}"
elif selected_cu == "Todos los Centros":
    filtro = df_carreras["CARRERA"] == selected_carrera
    etiqueta = f"Carrera: {selected_carrera}"
else:
    filtro = (df_carreras["CU"] == selected_cu) & (df_carreras["CARRERA"] == selected_carrera)
    etiqueta = f"{selected_carrera} en {selected_cu}"

df_ts = (
    df_carreras[filtro]
    .groupby("ds")[["ASPIRANTES", "ADMITIDOS"]]
    .sum()
    .reset_index()
)

if df_ts.empty:
    print(f"No hay datos históricos para {etiqueta}.")
    return

df_asp = df_ts.rename(columns={"ASPIRANTES": "y"})[["ds", "y"]]
df_adm = df_ts.rename(columns={"ADMITIDOS": "y"})[["ds", "y"]]

m_asp = train_prophet_model(df_asp)
m_adm = train_prophet_model(df_adm)

fcst_asp = predict_prophet(m_asp, int(periods_to_predict))
fcst_adm = predict_prophet(m_adm, int(periods_to_predict))

print(f"🧠 Aspirantes – {etiqueta}")
display(fcst_asp.head(int(periods_to_predict)))
print(f"🧠 Admitidos – {etiqueta}")
display(fcst_adm.head(int(periods_to_predict)))

m_asp.plot(fcst_asp); plt.title(f"Aspirantes • {etiqueta}"); plt.show()
m_adm.plot(fcst_adm); plt.title(f"Admitidos • {etiqueta}"); plt.show()

```

En esta última parte se construye la interfaz visual:

Se definen tres controles:

- Dropdown de Centro Universitario (CU)
- Dropdown de Carrera
- Slider para elegir el número de semestres a predecir

Se conectan estos controles a la función `plot_predictions` usando `ipywidgets.interactive()`.

Cada vez que el usuario cambia una selección, la función se ejecuta automáticamente y actualiza los resultados y gráficos.

```

interactive_plot = interactive(
    plot_predictions,
    selected_cu=cu_dropdown,
    selected_carrera=carrera_dropdown,
    periods_to_predict=periods_slider,
)

display(interactive_plot)

```

Centro:

Carrera:

Semestres:  4.00

Centro:

Carrera:

Semestres:  4.00

UDG Total – Próximos 4 semestres

## > Modelado Predictivo de Tasa de Admisión

↳ 10 cells hidden

## > Clasificación de Carreras o Centros según Competitividad con K-Means Clustering

↳ 14 cells hidden

## ✓ Detección de Anomalías en la Demanda y Admisión

Detectar registros atípicos entre las carreras de la Universidad de Guadalajara, considerando su comportamiento histórico en demanda, admisión y nivel de exigencia. Esto permite identificar eventos fuera de lo normal, posibles errores de captura, o transformaciones institucionales significativas.

Modelos utilizados:

- Isolation Forest (Bosque de Aislamiento)
- Local Outlier Factor (LOF)

Ambos algoritmos permiten detectar puntos que se comportan de manera estadísticamente diferente al resto de los datos, sin necesidad de etiquetas previas.

Variables utilizadas como entrada:

- ASPIRANTES
- ADMITIDOS
- PUNTAJE\_MINIMO
- TASA\_DEMANDA
- TASA\_ADMISION

(Todas calculadas como promedios por carrera, desde df\_carreras\_avg)

Resultado esperado:

- Etiquetar automáticamente cada carrera como:
- Normal (comportamiento esperado)
- Anómala (comportamiento fuera de lo común)

Visualización de las anomalías en el espacio PCA (reducción de dimensiones), para facilitar su interpretación.

En esta etapa se construye la matriz de características que alimentará a los modelos de detección. Se seleccionan las variables numéricas más relevantes para caracterizar la competitividad y el comportamiento histórico de cada carrera:

- ASPIRANTES,
- ADMITIDOS,
- PUNTAJE\_MINIMO,
- TASA\_DEMANDA,
- TASA\_ADMISION

(todas como promedios por carrera).

Posteriormente, los datos son normalizados utilizando StandardScaler, lo que transforma cada variable para que tenga media cero y desviación estándar uno. Este paso es esencial, ya que evita que variables con mayor escala (por ejemplo, número de aspirantes)

dominen el análisis frente a otras igualmente importantes, como la tasa de admisión.

```
from sklearn.preprocessing import StandardScaler

# Variables clave ya procesadas en df_carreras_avg (por carrera promedio)
features_anomalia = ["PUNTAJE_MINIMO", "TASA_DEMANDA", "TASA_ADMISION", "RELACION_AS_AD"]

df_anomalia = df_carreras_avg[features_anomalia].dropna().copy()

# Escalado
scaler_anomalia = StandardScaler()
X_scaled_anomalia = scaler_anomalia.fit_transform(df_anomalia)
```

Se entrena un modelo de Isolation Forest, un algoritmo basado en árboles binarios aleatorios que está diseñado específicamente para identificar outliers en datos multivariados. Su principio fundamental es que los datos anómalos tienden a ser más fáciles de aislar: requieren menos divisiones (splits) para separarlos del resto del conjunto en un árbol.

El modelo genera múltiples árboles de aislamiento y mide para cada punto la profundidad promedio necesaria para aislarlo. Si un dato es muy diferente de los demás, se necesitarán menos divisiones para aislarlo (es decir, estará más cerca de la raíz del árbol), lo cual se traduce en una profundidad baja.

- Si un punto tiene profundidad promedio baja → es considerado un outlier.
- Si tiene profundidad alta → es un dato normal, ya que está bien integrado en la estructura general de los datos.

Al final, el modelo etiqueta cada observación con:

- 1: comportamiento normal (difícil de aislar),

En esta etapa se aplica el modelo Local Outlier Factor (LOF) para complementar la detección realizada con Isolation Forest. A diferencia del enfoque global del paso anterior, LOF identifica anomalías locales, es decir, observaciones que presentan una densidad anormalmente baja comparada con sus vecinas más cercanas.

El modelo compara cada punto con su entorno local (por defecto, los 20 vecinos más cercanos) y calcula una relación de densidades. Si una carrera se encuentra en una zona donde sus características se alejan considerablemente de sus vecinas, su puntuación LOF será alta (desviación).

- $LOF \approx 1$  → comportamiento normal (densidad similar a su entorno)
- $LOF \gg 1$  → posible outlier (densidad mucho menor a la esperada)

El modelo clasifica con -1 a los registros que se desvían significativamente (outliers), y con 1 a los normales.

*Esto permite detectar carreras que, aunque no sean extremas en términos globales, sí tienen un comportamiento atípico dentro de su contexto inmediato (por ejemplo, carreras que tienen una tasa de admisión alta a pesar de una demanda considerable).*

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

# Modelo Isolation Forest
model_iforest = IsolationForest(contamination='auto', random_state=42)
df_anomalia["ISO_ANOMALY"] = model_iforest.fit_predict(X_scaled_anomalia) # -1 = anómalo

# Modelo LOF
model_lof = LocalOutlierFactor(n_neighbors=20, contamination='auto')
df_anomalia["LOF_ANOMALY"] = model_lof.fit_predict(X_scaled_anomalia) # -1 = anómalo

# Anomalía combinada (por lo menos uno)
df_anomalia["ANOMALIA"] = (df_anomalia["ISO_ANOMALY"] == -1) | (df_anomalia["LOF_ANOMALY"] == -1)
```

Para incrementar la robustez del análisis, se combinan los resultados de ambos modelos. Una carrera es clasificada como anómala si al menos uno de los dos algoritmos la detectó como tal (-1).

Este enfoque de combinación tiene dos beneficios clave:

1. Reducción de falsos negativos: se detectan más casos relevantes.
2. Mayor confiabilidad: se consideran tanto outliers globales como locales.

El resultado final es una nueva columna booleana (ANOMALIA) que indica si una carrera debe ser examinada con atención.

La tabla presenta las 11 carreras que fueron clasificadas como anómalas por al menos uno de los modelos (ISO\_ANOMALY = -1 o LOF\_ANOMALY = -1). Para cada una se incluyen sus valores promedio en las variables clave:

- PUNTAJE\_MINIMO
- TASA\_DEMANDA
- TASA\_ADMISION
- RELACION\_AS\_AD

Además, se indica qué modelo detectó la anomalía:

- "ISO\_ANOMALY" = -1 → fue marcada por Isolation Forest
- "LOF\_ANOMALY" = -1 → fue marcada por LOF
- Si ambos son -1 → alta concordancia entre modelos, anomalía robusta

Ejemplos destacados:

LICENCIATURA EN DISEÑO, ARTE Y TECNOLOGÍAS INTERACTIVAS: Presenta una tasa de demanda muy elevada (15.69) con una tasa de admisión baja (0.14) y una relación aspirantes/admitidos extremadamente alta (14.24). Ambos modelos coinciden en clasificarla como anómala.

LICENCIATURA EN ENFERMERÍA (NIVELACIÓN) y GEOGRAFÍA: Son detectadas por LOF como outliers a pesar de tener tasas de demanda bajas, posiblemente por su tasa de admisión inusualmente alta (~0.97), lo que contrasta con el resto del conjunto.

MEDICO CIRUJANO Y PARTERO: Combinación extrema de alta demanda (8.32), baja tasa de admisión (0.15) y alta relación aspirantes/admitidos (8.54). Aislada rápidamente por Isolation Forest y también marcada por LOF.

```
anomalas = df_anomalia[df_anomalia["ANOMALIA"]].copy()
print(f"Total de carreras con comportamiento anómalo: {anomalas.shape[0]}")
display(anomalas[features_anomalia + ["ISO_ANOMALY", "LOF_ANOMALY"]])
```

Total de carreras con comportamiento anómalo: 11

|  | TIPO                                                    | PUNTAJE_MINIMO | TASA_DEMANDA | TASA_ADMISION | RELACION_AS_AD | ISO_ANOMALY | LOF_ANOMALY |  |
|--|---------------------------------------------------------|----------------|--------------|---------------|----------------|-------------|-------------|--|
|  | CARRERA                                                 |                |              |               |                |             |             |  |
|  | INGENIERIA BIOMEDICA                                    | 166.202910     | 3.485404     | 0.312826      | 3.400751       | -1          | 1           |  |
|  | LICENCIATURA EN ARQUITECTURA                            | 159.304739     | 5.086740     | 0.207667      | 4.968320       | -1          | 1           |  |
|  | LICENCIATURA EN CRIMINOLOGIA                            | 133.380943     | 3.839206     | 0.310891      | 3.823622       | -1          | 1           |  |
|  | LICENCIATURA EN DISEÑO, ARTE Y TECNOLOGIAS INTERACTIVAS | 166.449492     | 15.691190    | 0.144410      | 14.244898      | -1          | -1          |  |
|  | LICENCIATURA EN ENFERMERIA (NIVELACION)                 | 73.322010      | 1.200211     | 0.944589      | 1.157385       | -1          | -1          |  |
|  | LICENCIATURA EN GEOGRAFIA                               | 100.146910     | 1.026161     | 0.978030      | 1.035681       | -1          | 1           |  |
|  | LICENCIATURA EN NUTRICION                               | 147.105803     | 5.101558     | 0.254332      | 4.985710       | -1          | 1           |  |
|  | LICENCIATURA EN PSICOLOGIA                              | 158.054523     | 6.065356     | 0.170791      | 6.079470       | -1          | -1          |  |

Para facilitar la interpretación visual de los resultados, se aplica Análisis de Componentes Principales (PCA), que permite reducir las múltiples dimensiones del análisis a solo dos (PCA 1 y PCA 2).

Cada carrera es proyectada como un punto en el plano 2D, y su color representa si fue o no clasificada como anómala.

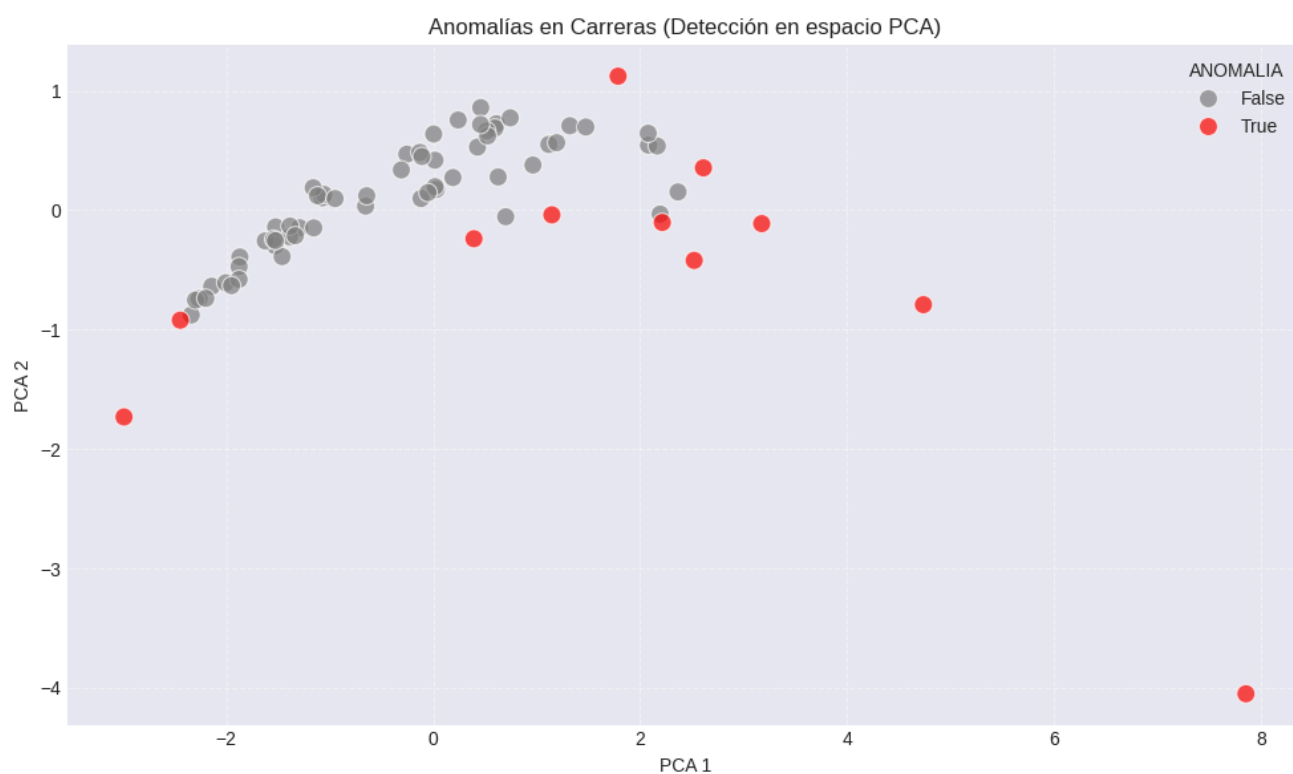
- Los puntos grises indican comportamiento normal.
- Los puntos rojos representan carreras con patrones atípicos, detectadas por al menos uno de los modelos.

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns

# Aplicar PCA si no se ha hecho antes
pca_anom = PCA(n_components=2, random_state=42)
X_pca_anom = pca_anom.fit_transform(X_scaled_anomalia)
```

```
# DataFrame visual
df_anomalia_vis = df_anomalia.copy()
df_anomalia_vis[["PCA1", "PCA2"]] = X_pca_anom

# Plot
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df_anomalia_vis,
    x="PCA1", y="PCA2",
    hue="ANOMALIA",
    palette={True: "red", False: "gray"},
    alpha=0.7,
    s=100
)
plt.title("Anomalías en Carreras (Detección en espacio PCA)")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show() # Muestra la figura (esto va DESPUÉS de guardarla)
```



Cada punto en el plano representa una **carrera universitaria**, descrita por sus valores estandarizados de:

- Puntaje mínimo,
- Tasa de demanda,
- Tasa de admisión,
- Relación aspirantes/admitidos.

Los dos ejes (**PCA 1**) y (**PCA 2**) son combinaciones lineales de estas variables y capturan la mayor parte de la **variabilidad del conjunto de datos**.

- El **color de cada punto** indica si fue detectado como **anómalo (True)** o **normal (False)**:
  - **Rojo**: carrera identificada como anómala (por Isolation Forest o LOF).
  - **Gris**: carrera con comportamiento esperado (normal).

Las carreras anómalas (rojas) tienden a ubicarse en regiones **más dispersas o periféricas** del espacio PCA, lo que indica que sus combinaciones de características son poco comunes respecto al resto del conjunto.

Algunas anomalías aparecen **aisladas visualmente** del resto del grupo, reforzando la validez de su detección.

También se observan **algunas anomalías integradas visualmente** en el grupo general, lo cual es coherente con lo que detecta LOF (anomalías locales, no necesariamente globales).

```
# Crear df_promedios desde el dataframe general ya cargado (df)
df_promedios = df.copy()

# Detectar columnas relevantes
cols_asp = [col for col in df_promedios.columns if 'AS' in col]
cols_adm = [col for col in df_promedios.columns if 'AD' in col]
cols_pm = [col for col in df_promedios.columns if 'PM' in col]

# Calcular promedios por carrera
# Asegurar que todas las columnas relevantes sean numéricas
df_promedios[cols_asp] = df_promedios[cols_asp].apply(pd.to_numeric, errors='coerce')
df_promedios[cols_adm] = df_promedios[cols_adm].apply(pd.to_numeric, errors='coerce')
df_promedios[cols_pm] = df_promedios[cols_pm].apply(pd.to_numeric, errors='coerce')

# Calcular promedios
df_promedios['ASPIRANTES_PROM'] = df_promedios[cols_asp].mean(axis=1, skipna=True)
df_promedios['ADMITIDOS_PROM'] = df_promedios[cols_adm].mean(axis=1, skipna=True)
df_promedios['PM_PROM'] = df_promedios[cols_pm].mean(axis=1, skipna=True)

# Interfaz interactiva con ipywidgets
import ipywidgets as widgets
from IPython.display import display, clear_output
import matplotlib.pyplot as plt
import seaborn as sns

# Lista de centros únicos
centros_disponibles = sorted(df_promedios["CU"].dropna().unique())

# Dropdown para seleccionar centro
dropdown_cu = widgets.Dropdown(
    options=centros_disponibles,
    value="CUCEI",
    description="Centro:",
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='50%')
)

# Función para graficar el top
def graficar_top_carreras(centro):
    clear_output(wait=True)
    display(dropdown_cu)

    df_centro = df_promedios[df_promedios["CU"] == centro][['CARRERA', 'ASPIRANTES_PROM', 'ADMITIDOS_PROM', 'PM_PROM']]

    # Top 10 para cada métrica
    top_asp = df_centro.sort_values(by='ASPIRANTES_PROM', ascending=False).head(10)
    top_adm = df_centro.sort_values(by='ADMITIDOS_PROM', ascending=False).head(10)
    top_pm = df_centro.sort_values(by='PM_PROM', ascending=False).head(10)

    # Crear gráficas
    fig, axs = plt.subplots(1, 3, figsize=(18, 8))

    sns.barplot(data=top_asp, y='CARRERA', x='ASPIRANTES_PROM', ax=axs[0])
    axs[0].set_title(f"Aspirantes promedio en {centro}")

    sns.barplot(data=top_adm, y='CARRERA', x='ADMITIDOS_PROM', ax=axs[1])
    axs[1].set_title(f"Admitidos promedio en {centro}")

    sns.barplot(data=top_pm, y='CARRERA', x='PM_PROM', ax=axs[2])
    axs[2].set_title(f"Puntaje mínimo promedio en {centro}")

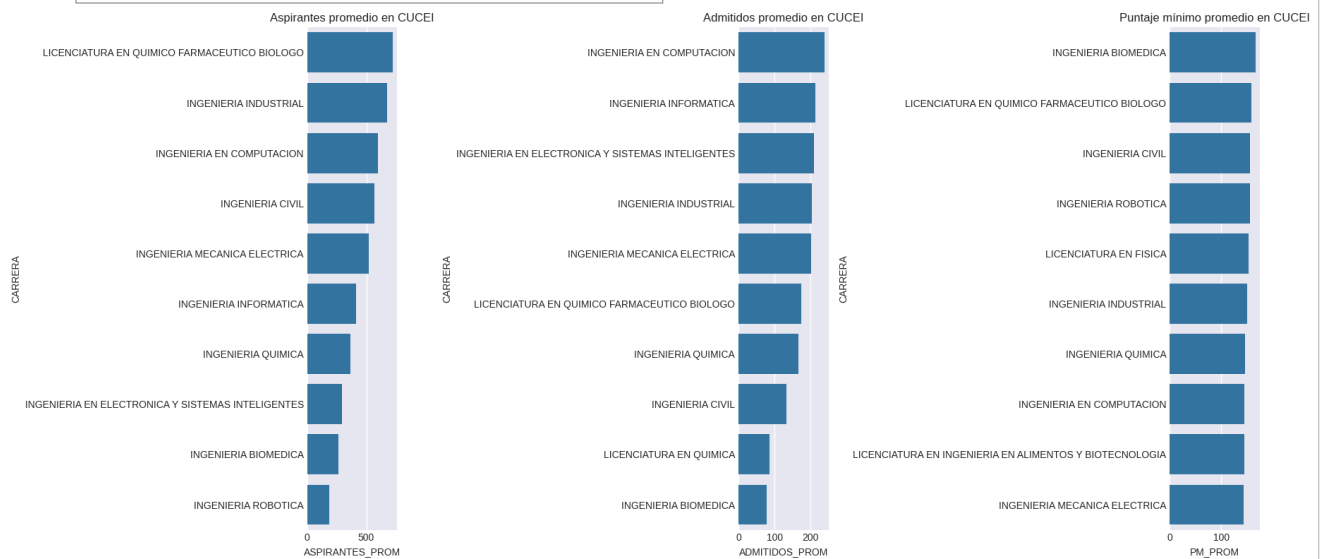
    plt.tight_layout()
    plt.show()

# Conectar dropdown con función
interactive_output = widgets.interactive_output(graficar_top_carreras, {'centro': dropdown_cu})

# Mostrar interfaz
display(dropdown_cu, interactive_output)
```

Centro: CUCEI

Centro: CUCEI



```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
import matplotlib.pyplot as plt

# Cargar datos
df = df[df['CU'] == 'CUCEI'].reset_index(drop=True)

# Extraer columnas de interés
cols_pm = [col for col in df.columns if 'PM' in col]
cols_as = [col for col in df.columns if 'AS' in col]
cols_ad = [col for col in df.columns if 'AD' in col]

# Crear variables resumen
df_proc = pd.DataFrame()
df_proc['CARRERA'] = df['CARRERA']
df_proc['media_aspirantes'] = df[cols_as].mean(axis=1)
df_proc['media_admitidos'] = df[cols_ad].mean(axis=1)
df_proc['media_pm'] = df[cols_pm].mean(axis=1)
df_proc['std_pm'] = df[cols_pm].std(axis=1)
df_proc['tasa_admision'] = df_proc['media_admitidos'] / df_proc['media_aspirantes']
df_proc['ratio_demand'] = df_proc['media_aspirantes'] / df_proc['media_admitidos']
df_proc = df_proc.replace([np.inf, -np.inf], np.nan).dropna()

# Guardar nombres para graficar
carreras = df_proc['CARRERA'].values
df_num = df_proc.drop(columns=['CARRERA'])

# Escalar datos
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_num)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Varianza explicada

```



```

explained = pca.explained_variance_ratio_ * 100
print(f"Varianza explicada: {explained[0]:.1f}% y {explained[1]:.1f}%")

# Isolation Forest
iso = IsolationForest(random_state=42)
iso_preds = iso.fit_predict(X_scaled)

# Local Outlier Factor
lof = LocalOutlierFactor()
lof_preds = lof.fit_predict(X_scaled)

# Clasificación final de anomalías
anomalies = (iso_preds == -1) | (lof_preds == -1)

# Diccionario de traducción al inglés
traductor = {
    "INGENIERIA EN COMPUTACION": "Computer Engineering",
    "INGENIERIA EN ELECTRONICA Y SISTEMAS INTELIGENTES": "Electronics and Intelligent Systems Eng.",
    "INGENIERIA INDUSTRIAL": "Industrial Engineering",
    "INGENIERIA BIOMEDICA": "Biomedical Engineering",
    "INGENIERIA EN CIENCIA DE MATERIALES": "Materials Science Engineering",
    "LICENCIATURA EN FISICA": "Physics",
    "LICENCIATURA EN QUIMICA": "Chemistry",
    "LICENCIATURA EN MATEMATICAS": "Mathematics",
    "INGENIERIA CIVIL": "Civil Engineering",
    "INGENIERIA INFORMATICA": "Informatics Engineering",
    "INGENIERIA EN LOGISTICA Y TRANSPORTE": "Logistics and Transport Engineering",
    "INGENIERIA MECANICA ELECTRICA": "Mechanical-Electrical Engineering",
    "INGENIERIA FOTONICA": "Photonics Engineering",
    "LICENCIATURA EN INGENIERIA EN ALIMENTOS Y BIOTECNOLOGIA": "Food and Biotechnology Engineering",
    "LICENCIATURA EN QUIMICO FARMACEUTICO BIOLOGO": "Pharmaceutical Biologist Chemist",
    "INGENIERIA EN TOPOGRAFIA GEOMATICA": "Geospatial Surveying Engineering",
    "INGENIERIA QUIMICA": "Chemical Engineering",
    "INGENIERIA ROBOTICA": "Robotics Engineering"
}

# Traducir nombres
df_proc['CARRERA_EN'] = df_proc['CARRERA'].map(traductor).fillna(df_proc['CARRERA'])
carreras = df_proc['CARRERA_EN'].values

plt.figure(figsize=(10, 7))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c='skyblue', edgecolor='k', s=70)
plt.title(f"PCA of CUCEI STEM Programs\n(PC1={explained[0]:.1f}%, PC2={explained[1]:.1f}%)", fontsize=16)
plt.xlabel("Principal Component 1", fontsize=16)
plt.ylabel("Principal Component 2", fontsize=16)
for i, name in enumerate(carreras):
    plt.text(X_pca[i, 0]+0.05, X_pca[i, 1]+0.05, name, fontsize=12, alpha=0.6)
plt.grid(True)
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 7))
for i in range(len(X_pca)):
    color = 'red' if anomalies[i] else 'gray'
    plt.scatter(X_pca[i, 0], X_pca[i, 1], c=color, edgecolor='k', s=70)

plt.title(f"Anomaly Detection in CUCEI STEM Programs\n(ISO/LOF ensemble)", fontsize=16)
plt.xlabel("Principal Component 1", fontsize=16)
plt.ylabel("Principal Component 2", fontsize=16)
for i, name in enumerate(carreras):
    if anomalies[i]:
        plt.text(X_pca[i, 0]+0.05, X_pca[i, 1]+0.05, name, fontsize=12, alpha=0.7, color='red')
plt.grid(True)
plt.tight_layout()
plt.show()

```

